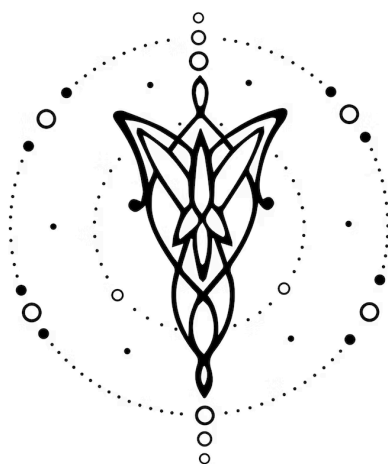




Especificación

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

v1.0.0

1. Equipo	1
2. Repositorio	1
3. Dominio	2
4. Construcciones	2
5. Casos de Prueba	3
6. Ejemplos	3

1. Equipo

Nombre	Apellido	Legajo	E-mail
Nicolas	Rossi	53.225	nrossi@itba.edu.ar

2. Repositorio

La solución y su documentación serán versionadas en: [JS-Minifier](#).

3. Dominio

Desarrollar un minificador de Javascript que, aunque simple, buscará reducir el tamaño del código mediante una serie de reglas, progresando desde transformaciones textuales básicas hasta optimizaciones semánticas más avanzadas.

Se espera que la implementación sea iterativa, y que demuestre la comprensión de los principios del compilador, específicamente el análisis léxico, el análisis sintáctico mediante la construcción de un AST y los fundamentos del análisis semántico para la optimización de código dentro de un subconjunto definido del lenguaje Javascript.

4. Construcciones

El lenguaje desarrollado debería ofrecer las siguientes construcciones, prestaciones y funcionalidades:

- (I). La eliminación de espacios en blanco no esenciales (espacios, tabulaciones, saltos de línea) y la eliminación de comentarios del código de entrada.
- (II). Transformación de expresiones constantes (Constant folding).
- (III). Acortamiento de los nombres de variable garantizando que permanezcan únicos dentro de su alcance.
- (IV). Eliminación de símbolos llaves y paréntesis innecesarios.
- (V). Transformación de booleanos *true/false* a su forma minificada *!0/!1*.
- (VI). Minificación de grandes números usando notación científica.
- (VII). Minificación de sentencias *if/if-else* haciendo uso de operadores..
- (VIII). Minificación de los constructores de tipo. (Eg. *Number(x) → +x*).
- (IX). Eliminación de código duplicado.
- (X). Eliminación de código inaccesible.
- (XI). Las definiciones de variables aceptadas son haciendo uso de *let, const*.
- (XII). Se aceptarán funciones del tipo fecha y declaradas con la keyword *function*.
- (XIII). No serán aceptadas definiciones de clase (*class, constructor, extends*).
- (XIV). Serán aceptados los operadores matemáticos: *+, -, *, /, %, **, ++, -*
- (XV). Serán aceptados los operadores lógicos: *||, &&, !*
- (XVI). Serán aceptados los operadores de comparación: *==, !=, ===, !==, <, >, >=, <=*
- (XVII). Serán aceptados los operadores bitwise: *~, &, ^, @, ~, <<, >>, >>>*
- (XVIII). Serán aceptadas las estructuras de control: *for, if, while, switch, do..while, for..in, for..of, try..catch*

5. Casos de Prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

- (I). Un programa formado por combinación de estructuras de control mencionadas en la sección 4.
- (II). Un programa que contenga una función de flecha con un único argumento entre paréntesis.
- (III). Un programa que contenga un literal booleano.
- (IV). Un programa que contenga espacios innecesarios y comentarios.
- (V). Un programa que contenga expresiones matemáticas que puedan ser compactadas.
- (VI). Un programa que contenga expresiones booleanas que puedan ser compactadas.
- (VII). Un programa que contenga un constructor del tipo *Number(x)*.
- (VIII). Un programa que contenga código inaccesible.
- (IX). Un programa que contenga código duplicado.

Además, los siguientes casos de prueba de **rechazo**:

- (I). Un programa malformado.
- (II). Un programa que contenga definiciones de variables utilizando la palabra reservada *var*
- (III). Un programa que contenga definiciones del tipo *class*
- (IV). Un programa que no esté formado por una combinación de estructuras admitidas por la sección 4.

6. Ejemplos

Input:		Minified Output:
True;	→	!0;
"a" + "b";	→	"ab";
2 * 3;	→	6;
Infinity;	→	1/0;

Input:		Minified Output:
Number(x)	→	+x
Boolean(x)	→	!!x
String(x)	→	x+""

Input:

```
/* Just a lonely comment */
function foo() {
  // comment
  let longLocalVariableName = 5 - 4;
  if (longLocalVariableName) {
    console.log(longLocalVariableName);
  }
}
```

Minified Output:

```
function foo(){let a=1;if(a)console.log(a)}
```

Input:

```
function foo() {
  if (x) {
    a();
  }
}

function foo2() {
  if (x) a();
  else b();
}
```

Minified Output:

```
function foo(){x&&a()}function foo2(){x?a():b()}
```